



山东大学

信息科学与工程学院

2024 – 2025 学年第二学期

实 验 报 告

课程名称: 微处理器原理与应用

实验名称: 实验 2.1

数值转换作业 (计算机组成原理内容)

专 业 班 级 通信工程 二班

学 生 学 号 202400120045

学 生 姓 名 刘哲旭

实 验 时 间 2025 年 3 月 13 日

【实验目的】

1. 掌握原码、补码、反码、移码等概念。

【实验具体内容】

一、选择题（如果为计算题，写出简要的计算过程）

1、一个四位二进制补码的表示范围是（B）

A、0~15 B、-8~7 C、-7~7 D、-7~8

在四位二进制补码表示法中，最高位（即第4位）被用作符号位：当该位为0时，表示正数或零；当该位为1时，表示负数。其余三位用于表示数值的大小。

对于正数（包括零），符号位为0，后三位直接给出二进制值，从000到111，对应十进制0到7。

对于负数，符号位为1，后三位同样从000到111，但这些值采用补码规则进行解释：它们代表从-1到-7的范围，其中111表示-1、110表示-2、...、001表示-7，而000则表示-0（即0，与正数的000等价）。然而，四位补码有一个特殊情况：当符号位为1且后三位为000时，即1000，它被定义为-8，而不是-0。这样，四位补码能够完整表示从-8到+7的全部整数

2、十进制数-48 用补码表示为（B） A、10110000 B、11010000 C、11110000 D、11001111

首先写出其原码为1011 0000。接着，通过将原码中的除了最高位都取反可得到反码1100 1111，最后在反码的基础上加1，就能得到补码1101 0000。

3、如果 X 为负数，由[x]补求[-x]补是将（D）

A、[x]补各值保持不变 B、[x]补符号位变反，其他各位不变
C、[x]补除符号位外，各位变反，末位加1 D、[x]补连同符号位一起各位变反，末位加1

当 x 代表一个负数时，-x 则对应于 x 的绝对值，即一个非负数。在补码系统中，求一个数的相反数（即 [-x] 补）的通用规则是：

对 [x] 补整体取反（包括符号位），然后 末位加 1

4、机器数 80H 所表示的真值是-128，则该机器数为（C）形式的表示。

A、原码 B、反码 C、补码 D、移码

80H 换算成 8 位二进制数 10000000, 为-128 的补码

5、在浮点数中, 阶码、尾数的表示格式是 (A)。

- A、阶码定点整数, 尾数定点小数 B、阶码定点整数, 尾数定点整数 C、
阶码定点小数, 尾数定点整数 D、阶码定点小数, 尾数定点小数

阶码 (存储的指数部分) 是定点整数 (偏置整数形式)。尾数 (f 部分) 是定点小数 (规格化后为 1.f 的形式)。浮点数 = $(-1)^s \times (1 + f) \times 2^e$, 其中 s 代表符号位, f 是尾数的小数部分, e 则是阶码的实际值减去一个固定的偏移量。

6、已知 $[x]_{\text{补}} = 10110111$, $[y]_{\text{补}} = 01001010$, 则 $[x - y]_{\text{补}}$ 的结果是 (A)。

- A、溢出 B、01101010 C、01001010 D、11001010

由补码的运算规则得, $[x-y]_{\text{补}} = [x]_{\text{补}} + [-y]_{\text{补}}$ 。而 $[-y]_{\text{补}} = 10110101 + 1 = 10110110$ 。因此, $[x-y]_{\text{补}} = 10110111 + 10110110 = 101101101$, 发生溢出

7、某机字长 8 位, 含一位数符, 采用原码表示, 则定点小数所能表示的非零最小正数为 (D)。

- A、 2^{-9} B、 2^{-8} C、-1 D、 2^{-7}

八位减去一个符号位, 表示数值的一共七位。能表示的非零最小正数为 0.0000001, 即 2^{-7}

8、下列数中最小的数是 (C)。

- A、 $[10010101]_{\text{原}}$ B、 $[10010101]_{\text{反}}$ C、 $[10010101]_{\text{补}}$ D、 $[10010101]_2$

A: $[10010101]_{\text{原}} = -1 \times (1+4+16) = -21$

B: $[10010101]_{\text{反}} = [11101010]_{\text{原}} = -1 \times (2+8+32+64) = -106$

C: $[10010101]_{\text{补}} = [10010100]_{\text{反}} = [11101011]_{\text{原}} = -1 \times (1+2+8+32+64) = -107$

D: $1+4+16+128=149$

9、8 位补码表示的定点整数的范围是 (B) 。

- A、-128~+128 B、-128~+127 C、-127~+128 D、-127~+127

对于正数部分, 除去符号位, 表示数值的有七位, 能表示的最大正数为

$2^7-1=127$ ，对于负数部分，由于 10000000 在补码规则中表示-128，而不是-0，所以表示的最小负数为-128

10、已知 X 的补码为 10110100，Y 的补码为 01101010，则 X-Y 的补码为 (D)

A、01101010 B、01001010 C、11001010 D、溢出

由补码的运算规则得， $[x-y]_{\text{补}} = [x]_{\text{补}} + [-y]_{\text{补}}$ 。而 $[-y]_{\text{补}} = 10010101+1=10010110$ ， $[x-y]_{\text{补}}=101001010$ ，显然溢出。

11、将 -33 以单符号位补码形式存入 8 位寄存器中，寄存器中的内容为 (A)。

A、DFH

B、A1H C、5FH D、DEH

-33 的原码表示为 1010 0001，反码通过除符号位的取反得到 1101 1110。在此基础上加 1 得到补码 1101 1111，将其转换为十六进制，每四位对应一个十六进制数，得到结果为 DFH。

12、在机器数的三种表示形式中，符号位可以和数值位一起参加运算的是 (B)

A、原码 B、补码 C、反码 D、反码、补码

负数的补码 = 原码取反 +1。设计核心就是让符号位作为数值的一部分，直接与数值位按相同规则一起加减（进位自然处理）。

13、“溢出”一般是指计算机在运算过程是产生的 (C)。

A、数据量超过内存容量 B、文件个数超过磁盘目录区规定的范围

C、数据超过了机器的位所能表示的范围 D、数据超过了变量的表示范围

数据使用固定的位数（比如 8 位、16 位、32 位）来表示和存储。所谓“溢出”，就是指在进行算术运算（加、减、乘、除等）时，计算得到的结果超出了这个固定位数所能表示的数值范围，导致结果无法正确表达，出现表示错误或失真。

14、设有二进制数 $X=-1101110$ ，若采用 8 位二进制数表示，则 $[X]_{\text{补}}$ 的结果是 (D)

A、11101101 B、10010011 C、00010011 D、10010010

由负数得符号位为 1， $X_{\text{原}}=11101110$ ，补码为反码+1 得 $X_{\text{补}}=10010010$

15、假设有一个 16 机的某存储单元存放着数 1101101101001000，若该数作为原码表示十进制有符号整数（其中最高位为符号位）时，其值为 (B)。

A、-55510 B、-23368 C、-18762 D、56136

16、计算机内的数有浮点和定点两种表示方法。一个浮点法表示的数由两部分组成，即 (C)。

过程同第五题

A、11010010 B、10111011 C、10110110 D、10011010

18、十六进制数(AB)₁₆ 变换为等值的八进制数是 (A)。

先转化成二进制： $(AB)_{16} = (1010\ 1011)_2$ ，每三位二进制数换算成一个八进制数，换算结果为： $(253)_8$

A、 $(227)_8$ B、 $(1FF)_{16}$ C、 $(10100001)_2$ D、 $(1789)_{10}$

C: $2^7+2^5+1=161$ 所以最大的数是 D

20、十进制数 87 转换成二进制数是 (A)。

A、 $(1010111)_2$ B、 $(1101010)_2$ C、 $(1110011)_2$ D、 $(1010110)_2$

$$(87)_{10} = 1 \times 2^6 + 0 \times 2^5 + 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 1 \times 2 + 1 = (1010111)_2$$

21、十进制数 1385 转换成十六进制数为 (B)。

A、568 B、569 C、D85 D、D55

$$1385D = 5 \times 16^2 + 6 \times 16 + 9 = 569H$$

22、下列不同进制数中最大的数是 (D)。

A、10111001B B、257Q C、97D D、BFH

都换算成十进制：

$$A: 2^7 + 2^5 + 2^4 + 2^3 + 1 = 185$$

$$B: 2 \times 8^2 + 5 \times 8 + 7 = 175$$

$$D: 11 \times 16 + 15 \times 1 = 191$$

最大的数是 D

二、填空题 (如果为计算题，写出简要的计算过程)

1、已知 X、Y 为两个带符号的定点整数，它们的补码为： $[X]_{补} = 00010011B$ ， $[Y]_{补} = 11111001B$ ，则 $[X+Y]_{补} = \underline{00001100}_B$

在补码运算中， $[X]_{补} + [Y]_{补} = [X+Y]_{补}$ ， $00010011 + 11111001 = 1\ 00001100$ ，最高位发生溢出，结果位数超出原先固定位数的限制。

2、八位定点整数，采用二进制补码表示时，所能表示真值的十进制数的范围是 $\underline{-128 \sim +127}$ 。

过程同选择第九题

3、(09 年)已知 $[X]_{补} = 01110111B$ ， $[Y]_{补} = 01100010B$ ，则 $[X-Y]_{补} = \underline{00010101}$ 。

根据补码运算规则， $[X-Y]_{补} = [X]_{补} + [-Y]_{补}$ ， $[-Y]_{补} = 10011101$ ($[Y]$ 取反) $+1 = 10011110$ ，则 $[X-Y]_{补} = 01110111 + 10011110 = 1\ 00010101$ ，最高位发生溢出。

4、一个含有 6 个“1”、2 个“0”的八位二进制整数原码，可表示的最大数为 $\underline{7EH}$ 。

(用十六进制表示)

为使可表示的数最大，符号位为 0，剩下的 1 应该都在高位，所以可表示最大数的二进制形式为： $01111110B$ ，换算十六进制为 7EH

5、已知 $[X]_{补} = 10000000B$ ，则 $X = \underline{-128}$ 。

一个八位二进制数，当补码为 10000000，其值为-128

6、二进制数 10110000，若看成纯小数，且为补码，则其对应真值的十进制数是
____ -0.625 ____。

当把该二进制数看作纯小数形式时，最高位作为符号位，而小数点紧跟在符号位之后。10110000 的原码是 11010000，数值部分为 1010000。在这一数值位中，两个 1 分别对应于 2^{-1} 和 2^{-3} 的权值位置。因此，该二进制数所表示的数值为 $-1 \times 2^{-1} + 2^{-3} = -0.625$

7、数 x 的真值为-0.1011B，其原码表示为 ____ 1.1011B ____，补码表示为 ____ 1.0101B ____。

最高位为符号位，其余位为数值部分，不改变。故-0.1011B 原码表示 1.1011B，补码为反码+1，补码表示 1.0101B

8、十进制数 25.1875 对应的二进制数是 ____ 11001.0011B ____。

整数部分： $25 = 2^4 + 2^3 + 1$ ，小数部分 $0.1875 = 2^{-3} + 2^{-4}$

9、一个二进制整数右端加上三个零，形成的新数是原数的 ____ 8 ____ 倍

由位操作的规则得知，在二进制整数右端加上三个零，相当于左移 3 位，左移 1 位相当于数值 $\times 2$ ，则本题形成的新数是原数 8 倍

10、已知 $[X]_{\text{补}} = 10000000B$ ，则 $X =$ ____ -128 ____（十进制）

同填空第五题

11、已知 $[X]_{\text{补}} = 11111111$ ，X 对应的真值是 ____ -1 ____

反码为补码减一： $[X]_{\text{反}} = 11111110$ ，原码为 10000001，所以对应真值为-1

【实验心得】

这次实验发现主要是通过做题来完成，因为实验的发展需要具体的理论的指导。在解题的过程中我深刻体会到了这些基础知识的重要性与意义。无论是进制之间的转换、原码、反码和补码的转换等内容，都是必不可少的基础知识。在将来踏上工作岗位，如果知识停留在表面，只会操作但对这些底层的逻辑和细节不

熟悉，在遇到问题的时候可能会变得更加棘手。在今后的学习生活中，我要努力掌握和计算机相关的底层逻辑知识。

本次实验中，我的解题过程不是特别顺利，一些知识出现遗忘的情况，比如小数的进制转化等，以及机器码之间的转换也不够熟练，还好这次试验帮助我复习了在大一上学期 C 语言课程以及大二上学期的数电课程的相关知识，更进一步了解了各种进制之间的关系，同时也掌握了原码，反码和补码之间的转换和计算，重要的是我学习到补码运算的一些运算规则，明白了一个浮点法表示的数由阶码和尾数两部分组成。当然想要真正学习好计算机的相关原理和知识，这些远远不够。



1. 原码、反码和补码

(1) 原码是计算机中表示带符号数的一种最直观的方法。在原码表示中，最高位作为符号位：通常规定 0 表示正数，1 表示负数，其余各位用于表示数值的大小部分，数值部分与该数的二进制形式完全相同。。这种表示方法的优点是形式简单、容易理解，但缺点是存在两个零（+0 和 -0），并且在进行加减运算时处理起来较为复杂，因此在实际计算机运算中很少直接使用。

(2) 反码是在原码的基础上得到的一种表示方法，主要用于改进原码在运算方面的不足。对于正数来说，反码与原码完全相同；而对于负数来说，反码的求法是 符号位保持不变，其余数值位逐位取反（0 变 1，1 变 0）。反码在一定程度上简化了减法运算，但仍然存在 +0 和 -0 两种表示形式，因此在计算机系统中也不是最终采用的表示方法。

(3) 补码是在反码的基础上进一步改进得到的一种表示方法，也是现代计算机中最常用的带符号数表示方式。对于正数来说，补码与原码、反码都相同；而对于负数来说，补码的求法是 先求该数的反码，再在最低位加 1。例如，在 8 位表示中，+5 的补码为 00000101；-5 的原码为 10000101，反码为 11111010，再加 1 后得到补码 11111011。补码的最大优点是 只有一个零的表示形式，并且可以把减法运算转化为加法运算，从而大大简化了计算机硬件电路的设计，因此被广泛应用于计算机内部的数据表示和运算之中

原码、反码和补码是计算机中表示带符号整数的三种不同编码方式。它们在表示方法和运算规则上各有特点，其中补码由于在运算处理上更加简便、高效，并且适用于计算机内部的算术运算，因此成为实际系统中最常用的一种表示方式。

2. 二进制、八进制、十进制和十六进制

(1) **二进制**是以 2 为基数的计数系统，只使用 0 和 1 两个数字来表示数值。计算机内部的电路通常只有两种稳定状态（如高电平和低电平、开和关），因此二进制非常适合用于计算机的数据表示和运算。在二进制中，每一位的权值依次是 $2^0, 2^1, 2^2, 2^3$ 等。二进制是计算机系统最基本的数据表示形式。

(2) **八进制**是以 8 为基数的计数系统，使用 0~7 八个数字表示数值。在早期计算机系统中，八进制曾被广泛使用，因为 1 位八进制数正好对应 3 位二进制数，这样可以较为简洁地表示较长的二进制数。通过这种对应关系，可以方便地在二进制和八进制之间进行转换。

(3) **十进制**是人们日常生活中最常用的计数系统，以 10 为基数，使用 0~9 十个数字表示数值。十进制的每一位权值依次是 $10^0, 10^1, 10^2, 10^3$ 等。虽然计算机内部主要采用二进制进行运算，但在程序设计、数据输入输出以及人机交互过程中，十进制仍然是最常用的表示方式，因为它更符合人类的习惯。

(4) **十六进制**是以 16 为基数的计数系统，使用 0~9 和 A~F 共 16 个符号，其中 A、B、C、D、E、F 分别表示十进制的 10~15。十六进制与二进制之间也有非常方便的对应关系，因为 1 位十六进制数正好对应 4 位二进制数。由于这种转换简单，十六进制在计算机系统中经常用于表示内存地址、机器指令和调试信息等。

3. 阶码和尾数

阶码和尾数是计算机中表示浮点数（Floating-Point Number）时使用的两个重要部分。浮点数通常采用类似科学计数法的形式来表示一个数值，一般可以写成

$$N = M \times r^E$$

其中， M 表示**尾数**（Mantissa）， E 表示**阶码**（Exponent）， r 表示**基数**（在计算机中通常为 2）。尾数用于表示数值的有效数字部分，决定数值的精度；阶码用于表示数值的大小范围，决定小数点移动的位置。通过这种方式，可以用有限的位数表示非常大或非常小的数。

尾数（Mantissa）是浮点数中存储有效数字的部分。为了提高表示效率，尾数通常会进行**规格化处理**，即把小数点移动到固定位置，使最高有效位不为 0。例如，在二进制中，一个数可以表示为

$$0.1011_2 \times 2^3$$

这里的 0.1011 就是尾数，它包含了该数的主要有效数字，决定了数值的精确

程度。

阶码 (Exponent) 是浮点数中表示指数部分的内容，用来说明尾数需要乘以多少次基数。阶码实际上决定了小数点需要向左或向右移动多少位，从而确定数值的大小。例如，在表达式

$$0.1011_2 \times 2^3$$

中，3 就是阶码，表示尾数需要乘以 2^3 ，也就是将小数点向右移动 3 位。

举例说明：

假设有二进制数

$$1011.01_2$$

可以把它写成规格化形式：

$$0.101101_2 \times 2^4$$

在这个表示中：

尾数： 0.101101_2 （表示有效数字部分）

阶码： 4（表示小数点需要向右移动 4 位）

因此，通过**尾数表示精度**，通过**阶码表示数量级**，浮点数就能够在计算机中高效地表示非常大或非常小的数值。